

Denge — Normalizasyon Kontrolü

Kapsam: main dalı (5e43a88). Cihaz veritabanı (lib/data/db/tables.dart , şema v2), Supabase şeması (supabase/migrations/*.sql) ve shared_preferences anahtarları. Tablo ayrıntıları için bkz. docs/VERITABANI.md .

Yöntem: Her tablo için fonksiyonel bağımlılıklar (FD) çıkarıldı. Ardından tablolar 1NF, 2NF, 3NF ve BCNF kurallarına göre kontrol edildi. Sonra kodda bu bağımlılıkların gerçekten geçerli olup olmadığına bakıldı (ör. hedefler elle değiştirilebiliyor mu?).

1. Özet

Tablo	1NF	2NF	3NF	BCNF	Not
food_log_entries	✓	✓	✓*	✓*	*Besin değerleri kayıt anında kopyalanıyor. Bu bilinçli bir denormalizasyon (B4)
water_logs	✓	✓	✓	✓	İki aday anahtar var (id ve date), ikisi de anahtar olduğu için BCNF sağlanıyor
weight_entries	✓	✓	✓*	✓*	* date , measured_at 'ten türetilabiliyor (B5)
custom_foods	✓	✓	✓	✓	category için bulutta CHECK kısıtı yok (B9)
sync_state	✓	✓	✓	✓	
profiles (bulut)	⚠	✓	⚠	⚠	allergies dizi (B1). email ve weight_kg başka tablolarla tekrarlanıyor (B6, B7)
shared_preferences	⚠	-	⚠	-	İlişkisel değil. Liste ve JSON değerler var, favoriler başlığa bağlı (B2, B8)

Sonuç: Şema genel olarak **3NF düzeyinde** ve sağlam. Tek kolonlu UUID anahtarlar kullanıldığı için 2NF ihlali **mümkün değil**. Tespit edilen sorunların çoğu bilinçli tasarım kararı (geçmiş korumak, çevrimdışı çalışmak, senkron). Düzeltmesi önerilen, gerçek risk taşıyan bulgular: **B6** (e-posta tekrarı, şu an tutarsızlık üretebiliyor), **B1** (alerji dizisi) ve **B8** (favoriler).

Bulgu listesi

#	Bulgu	Kural	Önem	Öneri
B1	profiles.allergies bir text[] dizisi	1NF	Orta	Ayrı tabloya taşı veya CHECK ekle
B2	shared_preferences içinde listeler ve JSON	1NF	Düşük	Favorileri tabloya taşı (B8 ile birlikte)
B3	serving_label iki bilgiyi birlikte tutuyor (1 porsiyon (200 g))	1NF (zayıf)	Düşük	Gerekirse serving_grams sütunu ekle
B4	food_log_entries besin değerlerinin kopyasını tutuyor	3NF	—	Koru , bilinçli tarihsel kayıt
B5	date alanı logged_at / measured_at ile tekrarlanıyor	3NF	Düşük	Koru
B6	profiles.email = auth.users.email	3NF / tekrar	Yüksek	E-postayı tek kaynaktan oku
B7	profiles.weight_kg = son weight_entries.kg	3NF / tekrar	Düşük	Kod şu an tutarlı tutuyor, testle koru
B8	Favori tarifler başlığa göre, sadece cihazda	Anahtar seçimi	Orta	Kalıcı tarif kimliği ve favorite_recipes tablosu
B9	source_ref farklı tablolara işaret ediyor, category kısıtsız	Bütünlük	Düşük	CHECK ekle
B10	user_initials, user_name'den türetiliyor ama saklanıyor	3NF	Düşük	Sakla veya hesapla, ikisi de kabul edilebilir

2. Tablo tablo analiz

Gösterim: $X \rightarrow Y$ = "X bilinirse Y tek şekilde bilinir". Ortak senkron sütunları (created_at, updated_at, deleted_at, synced_at, server_updated_at) her tabloda yalnızca id'ye bağlı. Bu yüzden aşağıda tekrar yazılmadı.

2.1 food_log_entries

- **Anahtar:** id
- **FD:** id $\rightarrow$ date, meal, food_name, brand, serving_label, amount, kcal, protein_g, carbs_g, fat_g, source, source_ref, logged_at
- **1NF** ☒ Bütün değerler tek parça. (serving_label için bkz. B3.)
- **2NF** ☒ Anahtar tek sütun, kısmi bağımlılık olamaz.
- **3NF / BCNF** ☒* Anahtar olmayan bir sütunu belirleyen başka bir anahtar olmayan sütun aranırsa iki aday çıkar:
 - {food_name, brand, serving_label} $\rightarrow$ porsiyon başı değerler. Bu bağımlılık **geçerli değil**, çünkü katalog değişebilir ve aynı yiyecek farklı günlerde farklı değerlerle

kaydedilmiş olabilir. Kopyalamanın amacı da tam olarak bu (B4).

- `logged_at` → `date`. Yalnızca cihazın saat dilimi biliniyorsa geçerli, saat dilimi saklanmıyor (B5).

Bu yüzden tablo 3NF'yi sağlıyor. Yine de tasarım gereği tekrar eden veri içeriyor.

2.2 `water_logs`

- **Aday anahtarlar:** `id` ve `date` (cihazda). Bulutta `id` ve `(user_id, date)`.
- **FD:** `id` → `date`, `glasses`, `date` → `id`, `glasses`
- **BCNF** ✓ Her FD'nin sol tarafı bir aday anahtar.
- Not: Aynı gün iki cihazda farklı `id` ile oluşturulabiliyor. `upsert_water()` bunu `(user_id, date)` üzerinden birleştiriyor ve cihaz sunucunun `id`'sini benimsiyor. Bu doğru bir çözüm.

2.3 `weight_entries`

- **Anahtar:** `id`
- **FD:** `id` → `date`, `kg`, `measured_at`. `measured_at` → `date` (saat dilimi biliniyorsa, B5).
- Aynı gün birden fazla ölçüm olabildiği için `date` bir anahtar değil. Bu doğru.
- **3NF** ✓* (B5 dışında).

2.4 `custom_foods`

- **Anahtar:** `id`
- **FD:** `id` → `name`, `brand`, `serving_label`, `kcal_per_serving`, `protein_g`, `carbs_g`, `fat_g`, `category`, `barcode`
- `barcode` → `id`? **Hayır.** Aynı barkod birden fazla kez kaydedilebiliyor, barkod benzersiz değil. Bu yüzden ek bir bağımlılık yok.
- **BCNF** ✓

2.5 `sync_state`

- **Anahtar:** `entity`. **FD:** `entity` → `pulled_until`. **BCNF** ✓

2.6 `profiles` (Supabase)

- **Anahtar:** `user_id`
- **FD:** `user_id` → (bütün sütunlar)
- **1NF** ⚠ `allergies text[]` bir tekrar grubu (B1).
- **Hedefler (`calorie_goal`, `protein_goal_g`, ...) neden ihlal değil?** Bu değerler kurulumda `computeTargets(gender, age, height_cm, weight_kg, activity_level, weight_goal, weekly_pace_kg)` ile hesaplanıyor. Fakat Hedefler ekranından (`goals_screen.dart` → `updateGoals`) **elle değiştirilebiliyorlar**. Yani vücut bilgileri → hedefler bağımlılığı geçerli değil ve hedefler bağımsız veri. **İhlal yok.**
- **Tablolar arası tekrar** ⚠: `email` (B6) ve `weight_kg` (B7).

2.7 `shared_preferences` (anahtar-değer deposu)

İlişkisel bir yapı olmadığı için normal formlar doğrudan uygulanmaz. Yine de tasarım açısından:

- `favorite_recipes` (liste), `user_allergies` (liste), `notification_settings` (JSON) tek bir anahtarada birden fazla değer tutuyor (B2).
- `user_initials`, `user_name` 'den türetiliyor (B10).
- `user_weight`, `weight_entries` tablosundaki son ölçümün kopyası (B7 ile aynı konu).
- Profil verisinin cihazdaki kopyası `profiles` tablosunun **replikası**. Bu, normalizasyon sorunu değil, çevrimdışı çalışmanın gereği.

3. Bulgular ve öneriler

B1: `profiles.allergies` dizi olarak tutuluyor (1NF)

- **Sorun:** Bir sütunda birden fazla değer var. "Fıstık alerjisi olan kullanıcılar" gibi bir sorgu dizi operatörü gerektiriyor. Dizi içindeki değerlerin geçerli olup olmadığı da kontrol edilmiyor: veritabanı `'peanut'` yerine `'Peanut'` veya `'fistik'` yazılmasını engellemiyor. Uygulama tanımadığı değerleri okurken atlıyor (`AllergyProfile.decode`), yani hatalı bir değer sessizce kaybolur.
- **Öneri (tam normalize):**

```
create table public.profile_allergies (  
  user_id uuid not null references public.profiles (user_id) on delete cascade,  
  allergen text not null check (allergen in ('gluten','milk','egg','peanut','treeNut',  
                                             'soy','fish','shellfish','sesame')),  
  primary key (user_id, allergen)  
);
```

- **Pratik alternatif:** Alerjen listesi sabit ve kısa olduğu için diziyi bırakıp şu kısıtı eklemek de yeterli: `check (allergies <@ array['gluten','milk','egg','peanut','treeNut','soy','fish','shellfish','sesame']::text[])`. PostgreSQL'de bu yaygın ve kabul görmüş bir kullanım.

B2: `shared_preferences` içinde liste ve JSON değerler (1NF)

- Cihaz ayarları için sorun değil. **Kullanıcı verisi** olan favoriler ise buluta da gitmeli (B8). `notification_settings` cihaza özel, JSON olarak kalabilir.

B3: `serving_label` iki bilgiyi birlikte tutuyor

- `"1 porsiyon (200 g)"` hem birimi hem gramı içeriyor. Gram bilgisi sorgulanamıyor, yani "bugün kaç gram yedim" hesaplanamıyor.
- **Öneri:** Gram bazlı rapor gerekirse null olabilir bir `serving_grams REAL` sütunu ekle. Bu ekleyici bir değişiklik, migration kurallarına uyuyor. Şu an bir ihtiyaç yoksa dokunma.

B4: Günlük kayıtları besin değerlerinin kopyasını tutuyor (bilinçli)

- Yiyeceğe referans verip değerleri katalogdan okumak "daha normal" olurdu. Ama o zaman katalog güncellendiğinde **geçmiş günlerin kalorisi değişir**. Burada kopya, verinin kayıt anındaki halini saklıyor (snapshot), gereksiz bir tekrar değil.
- **Karar: koru.** Fatura satırında ürün fiyatının kopyalanması ile aynı mantık.

B5: `date` alanı zaman damgasından türetilabiliyor (bilinçli)

- `date`, kaydın yapıldığı andaki yerel saate göre belirleniyor. Saat dilimi saklanmadığı için `logged_at` 'ten sonradan güvenilir şekilde hesaplanamaz. Kullanıcı yurt dışına gitse bile kayıt doğru güne ait kalmalı. Ayrıca `date` indeksli ve günlük sorgularının temeli.
- **Karar: koru.**

B6: `profiles.email`, `auth.users.email` ile tekrarlanıyor

- **Risk (güncelleme anomalisi), şu an gerçekleşebiliyor:** Kişisel bilgiler ekranı (`personal_info_screen.dart`) e-postayı değiştirmeye izin veriyor. Ama bu değişiklik yalnızca `user_email` tercihinin ve `profiles.email` sütununa yazılıyor, Supabase Auth'taki giriş e-postası değişmiyor (`updateUser` yalnızca şifre için çağrılıyor). Sonuç: profilde bir e-posta görünüyor, giriş başka bir e-postayla yapılıyor. Ayarlar ekranı ise `Giriş yapıldı: ${account.email}` ile oturumdaki e-postayı gösteriyor, yani iki ekran farklı e-posta gösterebilir.
- **Öneri:** Hesap açıksa e-postayı **yalnızca oturumdan** (`auth.currentUser.email`) oku ve kişisel bilgiler ekranında e-posta alanını salt okunur yap. Ya da değişikliği `updateUser(email: ...)` ile Supabase Auth'a gönder (doğrulama e-postası gider). `profiles.email` sütununu hemen silme: sütun silmek migration kurallarına göre önce onay gerektiriyor. Önce kullanımı bitir, sonra kaldır.

B7: Güncel kilo üç yerde duruyor

- Aynı bilgi şu üç yerde: `weight_entries` (son ölçüm), `profiles.weight_kg` ve `shared_preferences: user_weight`.
- **Risk:** Biri güncellenip diğeri güncellenmezse profil ekranı ile kilo grafiği farklı kilo gösterir.
- **Mevcut durum: tutarlı.** Kilo yazan yollar (`logWeight` ve kurulumdaki `completeSetup`) ölçümü `weight_entries` tablosuna, `user_weight` tercihinin ve profile (`_touchProfile` → `profiles.weight_kg`) birlikte yazıyor. Uygulamada kilo kaydını silen bir yol yok.
- Migration yorumuna göre `profiles.weight_kg` bilinçli eklenmiş: ikinci cihaz, bütün kilo geçmişi inmeden önce doğru kiloyu görsün diye. Bu geçerli bir sebep.
- **Öneri:** Bu tutarlılığı bir testle koru: `logWeight` sonrası üç değer eşit olmalı. İleride kilo silme veya düzenleme eklenirse son ölçüm yeniden hesaplanıp profile yazılmalı. Bulutta isteğe bağlı olarak `weight_entries` üzerinden bir `latest_weight` view'i tanımlanabilir.

B8: Favori tarifler başlığa göre ve sadece cihazda

- Anahtar olarak değişebilen bir doğal anahtar (başlık) kullanılıyor. Tarif başlığı değişince favori kayboluyor. Favoriler buluta da gitmiyor, telefon değişince kayboluyor.
- **Öneri:** Tariflere değişmeyen bir kimlik ver (ör. `slug`) ve şu tabloyu ekle: `favorite_recipes(id, recipe_id, + ortak senkron sütunları)`, bulutta `unique (user_id, recipe_id)`.

B9: `source_ref` ve `category` için kısıt yok

- `source_ref`, `source` değerine göre bazen barkod, bazen `custom_foods.id` tutuyor. Bu yüzden gerçek bir FK tanımlanamıyor. Değerler kopyalandığı için bu kabul edilebilir, ama bağlantının doğruluğu yalnızca uygulama koduna emanet.

- Bulutta `custom_foods.category` için CHECK kısıtı yok (`meal` ve `source` için var). Cihazdaki SQLite tablolarında ise **hiç CHECK kısıtı yok**.
- **Öneri:** Bulutta `category` için `check (category in (...12 kategori...))` ekle. Cihaz tarafında doğrulama zaten repository katmanında yapılıyor.

B10: `user_initials` saklanıyor

- İsimden hesaplanabilen bir değer. İsim değişince baş harfler de yeniden yazılıyor (`_initials0f`), yani şu an tutarlı. Düşük öncelik.

4. Önerilen işlem sırası

1. **B6:** E-postayı oturumdan oku veya değişikliği Auth'a gönder (kod değişikliği, şema değişmiyor).
2. **B7:** Üç kilo değerinin eşitliğini test eden bir test ekle.
3. **B9:** `custom_foods.category` için CHECK ekleyen yeni bir migration.
4. **B1:** Alerjiler için CHECK kısıtı (pratik) veya `profile_allergies` tablosu (tam normalize).
5. **B8:** Tarif kimliği ve `favorite_recipes` tablosu (cihaz şema v3 + bulut migration).

B4 ve B5 bilinçli tasarım kararları, değiştirilmemeli. B2, B3 ve B10 ihtiyaç doğarsa ele alınabilir.